# UNIVERSIDAD TÉCNICA DE AMBATO

**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**
**CARRERA DE SOFTWARE**

**CICLO ACADÉMICO: AGOSTO 2026 – ENERO**

## I. PORTAD

| | |
|---|---|
| Tema: | Catálogo de la Biblioteca FISEI con árbol general y ABB en Java |
| Unidad de Organización Curricular: | BÁSICA |
| Nivel y Paralelo: | 3 - B |
| Alumnos participantes: | Tuza Quinatoa Edith Noemi |
| Asignatura: | Estructura de Datos |
| Docente: | Ing. Jose Caiza |

### 1. Actividades por desarrollar

1. Objetivo

Implementar en Java un árbol general n-ario y un Árbol Binario de Búsqueda (ABB) mediante programación orientada a objetos y recursividad, aplicando operaciones de inserción, búsqueda, recorridos, eliminación y análisis de su estructura.

2. Descripción de la actividad

**Caso práctico: catálogo de la Biblioteca FISEI.**

La Biblioteca se organiza en Áreas y cada Área en Estantes. Esta jerarquía se representa con un árbol general n-ario.

Cada Estante almacena sus libros en un ABB ordenado por el código del libro (entero positivo, sin repetidos).

El estudiante deberá desarrollar una aplicación Java que permita construir, consultar, recorrer y modificar ambas estructuras.

3. Estructura inicial

Construya la siguiente biblioteca. Los libros se insertan en el orden indicado:

```
BIB Biblioteca FISEI
├── SW Software
│   ├── SW-E1 Estante 1 de Software → 60, 35, 80, 20, 45, 70, 90, 40, 75
│   └── SW-E2 Estante 2 de Software → 15, 10, 25
├── TI Tecnologías de la Información
│   └── TI-E1 Estante 1 de TI → 50
└── TEL Telecomunicaciones
```

La biblioteca contiene 7 unidades, 4 hojas, una altura de 2 aristas y 13 libros.

4. Requerimientos de implementación

**Parte A. Enumeraciones y modelo**

- **TipoUnidad**: BIBLIOTECA, AREA y ESTANTE, con el método puedeTenerHijo(TipoUnidad hijo).
- **TipoRecorrido**: PREORDEN, INORDEN y POSORDEN.
- **Libro**: codigo (entero mayor que 0, sin setter), titulo, autor y anio (1900–2026), con validaciones.
- **UnidadBiblioteca**: codigo (en mayúsculas), nombre y tipo.

- **NodoUnidad**: dato, lista de hijos y un CatalogoABB (solo si es ESTANTE).
- **NodoLibro**: dato, izquierdo y derecho, con getters y setters.

**Parte B. Clase ArbolBiblioteca**

1. agregar(codigoPadre, nueva): agrega una unidad respetando BIBLIOTECA → AREA → ESTANTE, sin códigos repetidos.
2. buscar(codigo): busca una unidad sin distinguir mayúsculas; null si no existe.
3. altura(): altura del árbol en aristas.
4. contarHojas(): número de unidades sin hijos.
5. nivel(codigo): nivel de la unidad (raíz = 0); -1 si no existe.
6. mostrarJerarquia(): jerarquía en preorden, con 2 espacios por nivel.
7. contarLibrosTotal(): total de libros de todos los estantes.

**Parte C. Clase CatalogoABB**

1. insertar(libro): inserta por código; false si el código ya existe.
2. buscar(codigo): busca un libro; null si no existe.
3. contar(): número de libros del catálogo.
4. recorrer(tipo): recorrido en preorden, inorden o posorden.
5. trazaRecorrido(tipo): traza con el formato entra X / VISITA X / null -> regresa, 2 espacios por nivel y última línea “Secuencia TIPO: …”.
6. modificar(codigo, titulo, autor, anio): modifica todo menos el código; false si no existe.
7. eliminar(codigo): elimina hoja, nodo con un hijo y nodo con dos hijos (reemplazo por el sucesor inorden).
8. getRaiz() y setRaiz(raiz): acceso a la raíz.

**Parte D. Retos (en CatalogoABB)**

1. contarEnRango(a, b): cuenta los códigos entre a y b, con poda.
2. caminoBusqueda(codigo): códigos visitados desde la raíz hasta el libro.
3. esABBValido(): verifica todo el ABB con límites mínimo y máximo.
4. Extra (+0,15): static desdePreorden(int[] preorden): reconstruye el ABB a partir de su preorden, sin insertar uno por uno.

**Parte E. Programa principal**

Desarrollar un menú interactivo con Scanner que permita: cargar la estructura inicial; mostrar la jerarquía; agregar y buscar unidades; consultar altura, hojas, nivel y total de libros; insertar, buscar, modificar y eliminar libros de un estante; mostrar los recorridos con su traza; ejecutar los retos; y salir. Además, la clase PruebasBiblioteca deberá ejecutar mínimo 10 casos y mostrar X/N pruebas OK.

5. Casos de prueba obligatorios

El estudiante deberá demostrar:

- Biblioteca solo con la raíz: altura 0 y 1 hoja.

- Biblioteca inicial: altura 2, 4 hojas y 13 libros.
- Nivel de BIB, sw, SW-E2 y XYZ: 0, 1, 2 y -1.
- Agregar un ESTANTE como hijo de BIBLIOTECA: false.
- Recorridos de SW-E1 — preorden: 60 35 20 45 40 80 70 75 90; inorden: 20 35 40 45 60 70 75 80 90; posorden: 20 40 45 35 75 70 90 80 60.
- Insertar un código repetido: false.
- caminoBusqueda(42): [60, 35, 45, 40]. contarEnRango(40, 75): 5.
- Insertar 42 y eliminar 20, 70 y 60, en ese orden: preorden final 75 35 45 40 42 80 90.
- Buscar, modificar y eliminar un código inexistente: null / false.
- Crear un libro con datos inválidos: IllegalArgumentException.
- esABBValido() en un árbol con un nieto mal ubicado: false.

6. Organización del proyecto

El código deberá organizarse en cuatro paquetes:

```
src/
├── enums/    TipoUnidad.java, TipoRecorrido.java
├── modelo/   Libro.java, UnidadBiblioteca.java
├── negocio/  NodoUnidad.java, NodoLibro.java,
│             ArbolBiblioteca.java, CatalogoABB.java
└── app/      Main.java, DatosPrueba.java, PruebasBiblioteca.java
```

**Restricciones:** no se permite utilizar TreeSet, TreeMap, PriorityQueue, NavigableSet, NavigableMap, Collections.sort ni List.sort, ni ordenar listas para obtener recorridos. Todas las operaciones de los árboles deberán implementarse mediante recursividad.

7. Evidencias y entrega

- Repositorio público de GitHub Deber-Arboles-ApellidoNombre, con trabajo en la rama desarrollo unida a main mediante pull request.
- Al menos tres commits significativos; código, README.md y TRAZAS.md.
- PDF en Moodle con el enlace al repositorio, capturas de ejecución, capturas del historial de commits y del pull request, y la reflexión personal.
- Entrega: jueves 22 de octubre de 2026, 07:00, en Moodle (3 puntos).

**2. Solución propuesta**

Se modela el catálogo de la Biblioteca FISEI con dos estructuras recursivas organizadas en cuatro paquetes. El paquete enums define TipoUnidad (con la regla BIBLIOTECA → AREA → ESTANTE en puedeTenerHijo) y TipoRecorrido. El paquete modelo contiene Libro, que valida el código (mayor que 0, sin setter) y el año (1900–2026), y UnidadBiblioteca, que guarda el código en mayúsculas. El paquete negocio contiene los nodos y las dos clases principales: ArbolBiblioteca administra el árbol general n-ario de unidades, y cada NodoUnidad de tipo ESTANTE guarda un CatalogoABB, un ABB de libros ordenado por código. El paquete app contiene el menú con Scanner, los datos de la estructura inicial y las pruebas. Todas las operaciones de los árboles se implementan con recursividad, sin TreeSet, TreeMap, PriorityQueue, NavigableSet, NavigableMap, Collections.sort ni List.sort; los recorridos salen del orden natural del ABB y nunca de ordenar listas.

| Clase | Responsabilidad |
|---|---|
| TipoUnidad | Enumeración BIBLIOTECA, AREA, ESTANTE; puedeTenerHijo() fija qué tipo de unidad puede colgar de cuál. |
| TipoRecorrido | Enumeración PREORDEN, INORDEN y POSORDEN para recorridos y trazas. |
| Libro | Modelo del libro: codigo (> 0, sin setter), titulo, autor y anio (1900–2026); lanza IllegalArgumentException si los datos son inválidos. |
| UnidadBiblioteca | Modelo de una unidad: codigo (siempre en mayúsculas), nombre y tipo. |
| NodoUnidad | Nodo del árbol general: dato, lista de hijos y un CatalogoABB solo si la unidad es ESTANTE. |
| NodoLibro | Nodo del ABB: dato (Libro), hijo izquierdo y derecho, con getters y setters. |
| ArbolBiblioteca | Árbol general n-ario: agrega y busca unidades, calcula altura, hojas y nivel, muestra la jerarquía y suma los libros de todos los estantes. |
| CatalogoABB | ABB de libros: inserta, busca, cuenta, recorre, traza, modifica y elimina (tres casos); incluye los retos contarEnRango, caminoBusqueda, esABBValido y desdePreorden. |
| Main | Programa principal: menú interactivo con Scanner (12 opciones) con validación de la entrada numérica. |
| DatosPrueba | Carga la estructura inicial: 7 unidades y 13 libros. |
| PruebasBiblioteca | Ejecuta los casos de prueba obligatorios y muestra “X/N pruebas OK”. |

### 3. TDA

ArbolBiblioteca: raiz: NodoUnidad (unidad BIBLIOTECA; nunca es nula).

NodoUnidad: dato: UnidadBiblioteca,

hijos: List<NodoUnidad>,

catalogo: CatalogoABB (null si no es ESTANTE).

CatalogoABB: raiz: NodoLibro (null si el catálogo está vacío).

NodoLibro: dato: Libro,

izquierdo, derecho: NodoLibro.

Main: biblioteca: ArbolBiblioteca,

opcion: int (control del menú).

**Operaciones de ArbolBiblioteca:**

| Operación | Firma | Descripción |
|---|---|---|
| Agregar unidad | agregar(String codigoPadre, UnidadBiblioteca nueva) → boolean | Busca al padre y le agrega la unidad si el padre existe, el código no se repite y puedeTenerHijo() lo permite; si no, retorna false. |
| Buscar unidad | buscar(String codigo) → NodoUnidad | Recorre el árbol de forma recursiva sin distinguir mayúsculas; null si no existe. |
| Altura | altura() → int | 1 + la mayor altura de los hijos; una unidad sin hijos vale 0 (solo la raíz = 0). |

| Operación | Firma | Descripción |
|---|---|---|
| Contar hojas | contarHojas() → int | Una unidad sin hijos cuenta 1; si tiene hijos suma las hojas de cada hijo. |
| Nivel | nivel(String codigo) → int | Raíz = 0; baja sumando 1 por nivel; -1 si no existe. |
| Jerarquía | mostrarJerarquia() → String | Preorden con 2 espacios por nivel: código y nombre. |
| Total de libros | contarLibrosTotal() → int | Suma contar() del CatalogoABB de cada estante. |

### Operaciones de CatalogoABB:

| Operación | Firma | Descripción |
|---|---|---|
| Insertar | insertar(Libro libro) → boolean | Baja por la izquierda si el código es menor y por la derecha si es mayor; false si ya existe. |
| Buscar | buscar(int codigo) → Libro | Compara con el nodo actual y descarta un subárbol en cada paso; null si no existe. |
| Contar | contar() → int | 1 + libros del subárbol izquierdo + libros del derecho (0 si el nodo es null). |
| Recorrer | recorrer(TipoRecorrido tipo) → List<Libro> | Preorden (raíz, I, D), inorden (I, raíz, D) o posorden (I, D, raíz). |
| Traza | trazaRecorrido(TipoRecorrido tipo) → String | Escribe entra X, VISITA X y null -> regresa con 2 espacios por nivel y la línea final “Secuencia TIPO: …”. |
| Modificar | modificar(int codigo, String titulo, String autor, int anio) → boolean | Valida los datos nuevos y cambia todo menos el código; false si no existe. |
| Eliminar | eliminar(int codigo) → boolean | Hoja: se corta el enlace. Un hijo: el hijo ocupa su lugar. Dos hijos: se copia el sucesor inorden y se elimina de la derecha. |
| Contar en rango | contarEnRango(int a, int b) → int | Cuenta códigos entre a y b; poda el subárbol que no puede contener valores del rango. |
| Camino | caminoBusqueda(int codigo) → List<Integer> | Códigos visitados desde la raíz hasta el libro (o hasta donde terminaría la búsqueda). |
| Validar ABB | esABBValido() → boolean | Pasa límites mínimo y máximo hacia abajo; verifica todos los descendientes, no solo padre e hijo. |
| Extra | static desdePreorden(int[] preorden) → CatalogoABB | Reconstruye el ABB en O(n) con límites y un índice, sin insertar uno por uno. |

## 4. Relación entre requisitos y código

| Requisito | Aplicación |
|---|---|
| Enumeraciones y modelo (Parte A) | TipoUnidad con puedeTenerHijo(), TipoRecorrido, Libro (código sin setter, año 1900–2026), UnidadBiblioteca (código en mayúsculas), NodoUnidad y NodoLibro. |
| Árbol general n-ario | NodoUnidad guarda una lista de hijos; ArbolBiblioteca.agregar() valida jerarquía y códigos repetidos antes de añadir. |

| Requisito | Aplicación |
|---|---|
| Estructura inicial | DatosPrueba.cargarBiblioteca() crea las 7 unidades y carga los 13 libros en el orden indicado. |
| ABB por estante | Solo los NodoUnidad de tipo ESTANTE crean un CatalogoABB; el resto lo deja en null. |
| Recursividad en todas las operaciones | buscar, altura, contarHojas, nivel, insertar, contar, recorridos, traza, eliminar, retos y desdePreorden tienen caso base y se llaman a sí mismos sobre los hijos. |
| Eliminación con sucesor inorden | eliminar() copia el menor del subárbol derecho (minimo()) y luego lo elimina de ese subárbol. |
| Retos con poda y límites | contarEnRango() descarta ramas fuera del rango; esABBValido() propaga mínimo y máximo. |
| Menú con Scanner | Main: ciclo do-while con las opciones 1 a 12 y switch; leerEntero() repite la lectura hasta recibir un número válido. |
| Mínimo 10 pruebas | PruebasBiblioteca ejecuta 17 comprobaciones y muestra “17/17 pruebas OK”. |
| Restricciones del enunciado | No se usa TreeSet, TreeMap, PriorityQueue, NavigableSet, NavigableMap, Collections.sort ni List.sort; el orden sale de la estructura del ABB. |

## 5. Tipos de datos utilizados

| Tipo | Categoría | Variables |
|---|---|---|
| String | Referencia | codigo, nombre (UnidadBiblioteca); titulo, autor (Libro); codigoPadre, codigo (búsquedas) |
| int | Primitivo | codigo y anio (Libro); opcion, a, b, nivel; retorno de altura(), contarHojas(), contar(), nivel() y contarEnRango() |
| boolean | Primitivo | retorno de agregar(), insertar(), modificar(), eliminar(), esHoja(), esABBValido() y puedeTenerHijo() |
| Integer | Referencia (envoltorio) | min y max de esABBValido() (null = sin límite) |
| TipoUnidad, TipoRecorrido | Enumeración | tipo (UnidadBiblioteca), tipo (recorrer y trazaRecorrido) |
| Libro | Referencia (clase propia) | dato (NodoLibro), libro |
| UnidadBiblioteca | Referencia (clase propia) | dato (NodoUnidad), nueva |
| NodoUnidad | Referencia (clase propia) | raiz (ArbolBiblioteca), padre, hijo, nodo |
| NodoLibro | Referencia (clase propia) | raiz (CatalogoABB), izquierdo, derecho, sucesor |
| ArbolBiblioteca, CatalogoABB | Referencia (clase propia) | biblioteca (Main), catalogo (NodoUnidad) |
| List<NodoUnidad>, List<Libro>, List<Integer> | Referencia (colección) | hijos, resultado de recorrer(), camino de caminoBusqueda() |
| Scanner | Referencia | sc (entrada estándar) |

## 6. Diagrama de clases

Diagrama generado a partir del diseño real de la solución:

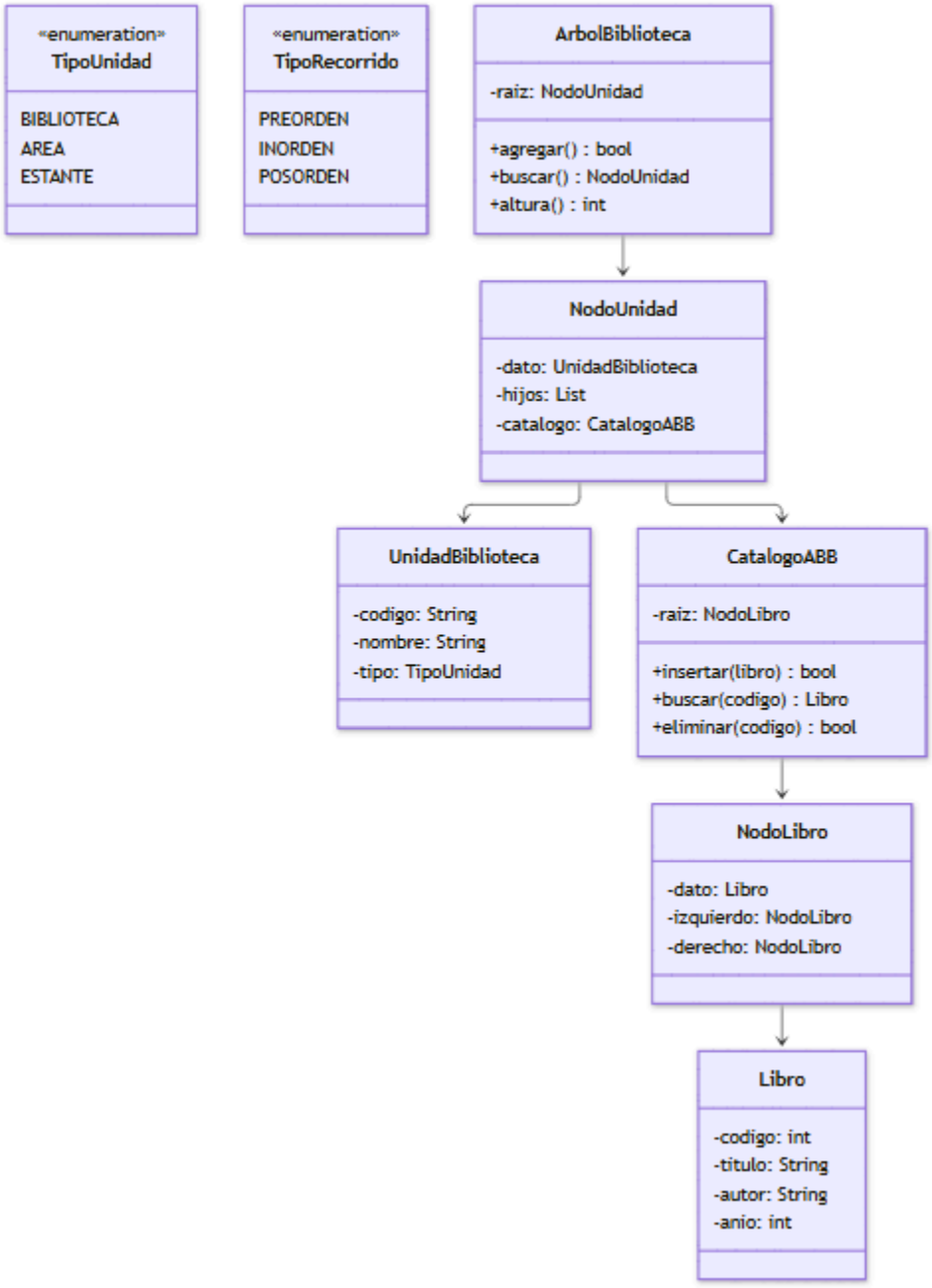
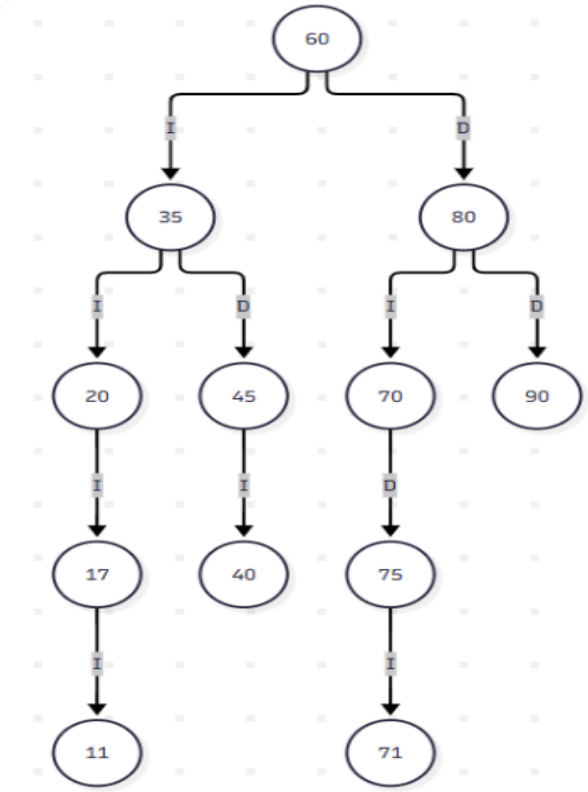


Diagrama de Árbol




## 7. Especificación del TDA

Estado: ArbolBiblioteca mantiene la referencia a la unidad raíz (BIBLIOTECA) de la que cuelgan listas de hijos; cada unidad de tipo ESTANTE guarda un CatalogoABB, que mantiene la referencia a la raíz de un ABB de libros donde cada NodoLibro tiene como máximo dos hijos.

| **Método / Operación** | **Resultado** |
|---|---|
| agregar(codigoPadre, nueva) | true si se añadió; false si el padre no existe, el código se repite o la jerarquía no lo permite. |
| buscar(codigo) (ArbolBiblioteca) | Retorna la unidad (sin distinguir mayúsculas) o null. |
| altura() / contarHojas() / nivel(codigo) | Altura en aristas (0 si solo existe la raíz); unidades sin hijos; nivel con raíz = 0 y -1 si no existe. |
| mostrarJerarquia() / contarLibrosTotal() | Texto en preorden con 2 espacios por nivel; suma de los libros de todos los estantes. |
| insertar(libro) | true si se insertó; false si el código ya existe. |
| buscar(codigo) (CatalogoABB) | Retorna el libro o null si no existe. |
| recorrer(tipo) / trazaRecorrido(tipo) | Lista de libros en el orden pedido; traza con entra, VISITA y null -> regresa más la línea “Secuencia TIPO: …”. |
| modificar(codigo, titulo, autor, anio) | true si existe y los datos son válidos; false si no existe; IllegalArgumentException si los datos son inválidos. |

| Método / Operación | Resultado |
|---|---|
| eliminar(codigo) | true si lo eliminó (hoja, un hijo o dos hijos con sucesor inorden); false si no existe. |
| contarEnRango / caminoBusqueda / esABBValido | Cantidad de códigos en [a, b]; lista de códigos visitados; true solo si todo el árbol respeta los límites. |

**Invariantes:**

- La jerarquía siempre cumple BIBLIOTECA → AREA → ESTANTE, y un ESTANTE nunca tiene hijos.
- Los códigos de unidad no se repiten (se comparan sin distinguir mayúsculas) y se guardan en mayúsculas.
- Solo los NodoUnidad de tipo ESTANTE tienen CatalogoABB; los demás lo tienen en null.
- En todo NodoLibro, los códigos de su subárbol izquierdo son menores y los de su subárbol derecho son mayores; no hay códigos repetidos.
- El código de un Libro es mayor que 0 y no se puede modificar (no existe setter).
- La altura se mide en aristas: un árbol con solo la raíz tiene altura 0.
- Todas las operaciones de los árboles son recursivas; no se usan estructuras ordenadas de Java ni se ordenan listas.

## 8. Pseudocódigo

```
INICIO
  biblioteca <- null ; opcion <- 0
  REPETIR
    mostrar menú (1..12)
    opcion <- leerEntero()      // repite hasta recibir un entero válido
    SEGUN opcion HACER
      1: biblioteca <- cargarBiblioteca()          // 7 unidades, 13 libros
      2: mostrar biblioteca.mostrarJerarquia()
      3: leer padre, código, nombre, tipo -> mostrar agregar(padre, nueva)
      4: leer código -> mostrar buscar(código) o "Unidad no encontrada"
      5: mostrar altura(), contarHojas(), contarLibrosTotal() y nivel(código)
      6..9: elegir estante -> insertar / buscar / modificar / eliminar libro
      10: elegir estante -> mostrar trazaRecorrido de los tres recorridos
      11: elegir estante -> contarEnRango, caminoBusqueda, esABBValido
      12: mostrar "Saliendo..."
      OTRO: mostrar "Opción no válida"
    FIN_SEGUN
  MIENTRAS opcion <> 12
FIN

FUNCION insertar(n, libro)
  SI libro.codigo = n.codigo RETORNAR falso
  SI libro.codigo < n.codigo ENTONCES
     SI n.izquierdo = null ENTONCES n.izquierdo <- nuevo(libro); RETORNAR verdadero
     RETORNAR insertar(n.izquierdo, libro)
  SI n.derecho = null ENTONCES n.derecho <- nuevo(libro); RETORNAR verdadero
  RETORNAR insertar(n.derecho, libro)

FUNCION eliminar(n, codigo)
  SI n = null RETORNAR null
  SI codigo < n.codigo ENTONCES n.izquierdo <- eliminar(n.izquierdo, codigo); RETORNAR n
```

```
  SI codigo > n.codigo ENTONCES n.derecho <- eliminar(n.derecho, codigo); RETORNAR n
  SI n.izquierdo = null RETORNAR n.derecho          // hoja o un hijo
  SI n.derecho = null RETORNAR n.izquierdo          // un hijo
  s <- minimo(n.derecho)                            // sucesor inorden
  n.dato <- s.dato
  n.derecho <- eliminar(n.derecho, s.codigo)
  RETORNAR n

FUNCION contarEnRango(n, a, b)
  SI n = null RETORNAR 0
  SI n.codigo < a RETORNAR contarEnRango(n.derecho, a, b)      // poda izquierda
  SI n.codigo > b RETORNAR contarEnRango(n.izquierdo, a, b)    // poda derecha
  RETORNAR 1 + contarEnRango(n.izquierdo, a, b) + contarEnRango(n.derecho, a, b)

FUNCION esValido(n, min, max)
  SI n = null RETORNAR verdadero
  SI (min <> null Y n.codigo <= min) O (max <> null Y n.codigo >= max) RETORNAR falso
  RETORNAR esValido(n.izquierdo, min, n.codigo) Y esValido(n.derecho, n.codigo, max)

FUNCION altura(u)                                   // árbol general
  m <- -1
  PARA CADA hijo DE u.hijos HACER m <- MAXIMO(m, altura(hijo))
  RETORNAR 1 + m

FUNCION contarHojas(u)
  SI u no tiene hijos RETORNAR 1
  RETORNAR suma de contarHojas(hijo) para cada hijo de u
```

### 9. Prueba de escritorio (traza manual con casos de prueba)

Se prueban los casos obligatorios del enunciado sobre el estante SW-E1 (insertado en el orden 60, 35, 80, 20, 45, 70, 90, 40, 75) y sobre la biblioteca inicial:

| Paso | Operación | Estado clave | Resultado |
|---|---|---|---|
| 1 | Biblioteca solo con la raíz | raiz = BIB; hijos = [] | altura() = 1 + max(-1) = 0; contarHojas() = 1. |
| 2 | Biblioteca inicial (opción 1) | BIB → SW, TI, TEL; SW → SW-E1, SW-E2; TI → TI-E1 | 7 unidades; 4 hojas (SW-E1, SW-E2, TI-E1, TEL); altura 2 (BIB → SW → SW-E1); 13 libros (9 + 3 + 1). |
| 3 | nivel("BIB"), nivel("sw"), nivel("SW-E2"), nivel("XYZ") | el código se normaliza con trim().toUpperCase() | 0, 1, 2 y -1. |
| 4 | agregar("BIB", ESTANTE) | BIBLIOTECA.puedeTenerHijo(ESTANTE) = false | false; el árbol no cambia. |
| 5 | Recorridos de SW-E1 | raíz 60; 35 (20, 45 (40)); 80 (70 (-, 75), 90) | Preorden: 60 35 20 45 40 80 70 75 90. Inorden: 20 35 40 45 60 70 75 80 90. Posorden: 20 40 45 35 75 70 90 80 60. |
| 6 | insertar(libro 60) repetido | 60 = raíz | false. |
| 7 | caminoBusqueda(42) | 60 → 35 → 45 → 40 (42 > 40 y 40 no tiene derecho) | [60, 35, 45, 40]. |
| 8 | contarEnRango(40, 75) | 60 en rango; 35 < 40 → solo su derecho; 80 > 75 → solo su izquierdo; 90 no se visita | Cuenta 40, 45, 60, 70 y 75 = 5. |

| Paso | Operación | Estado clave | Resultado |
|---|---|---|---|
| 9 | insertar 42; eliminar 20, 70 y 60 | 20 es hoja; 70 tiene un hijo (75); 60 tiene dos hijos: sucesor inorden = 75 | Preorden final: 75 35 45 40 42 80 90. |
| 10 | buscar(999), modificar(999, …), eliminar(999) | el código 999 no existe | null / false / false. |
| 11 | new Libro(-5, "Título", "Autor", 1800) | código ≤ 0 y año fuera de 1900–2026 | IllegalArgumentException. |
| 12 | esABBValido() con 50 → izq 30 → der 60 | 60 es mayor que 30 pero también mayor que el máximo 50 heredado | false (un chequeo solo padre-hijo no lo detectaría). |

**Salida esperada en consola (opción 2 con la estructura inicial):**

```
BIB Biblioteca FISEI
  SW Software
    SW-E1 Estante 1 de Software
    SW-E2 Estante 2 de Software
  TI Tecnologías de la Información
    TI-E1 Estante 1 de TI
  TEL Telecomunicaciones
```

## 10. Resultados obtenidos

La biblioteca inicial se construyó con 7 unidades, 4 hojas, una altura de 2 aristas y 13 libros, tal como exige el enunciado. Los recorridos de SW-E1 coincidieron con los esperados: preorden 60 35 20 45 40 80 70 75 90, inorden 20 35 40 45 60 70 75 80 90 y posorden 20 40 45 35 75 70 90 80 60. caminoBusqueda(42) devolvió [60, 35, 45, 40] y contarEnRango(40, 75) devolvió 5. Tras insertar 42 y eliminar 20 (hoja), 70 (un hijo) y 60 (dos hijos, sucesor inorden 75), el preorden final fue 75 35 45 40 42 80 90. Buscar, modificar y eliminar un código inexistente devolvió null/false, un libro con datos inválidos lanzó IllegalArgumentException y esABBValido() detectó el nieto mal ubicado. La clase PruebasBiblioteca mostró 17/17 pruebas OK y la salida del programa coincidió con la traza manual de la sección 9.

## Captura de ejecución: PruebasBiblioteca

```
[ OK ] 1. Biblioteca solo con la raíz: altura 0 y 1 hoja
[ OK ] 2. Biblioteca inicial: 7 unidades, altura 2, 4 hojas y 13 libros
[ OK ] 3. Nivel de BIB, sw, SW-E2 y XYZ: 0, 1, 2 y -1
[ OK ] 4. Agregar un ESTANTE como hijo de BIBLIOTECA: false
[ OK ] 4b. Agregar una unidad con código repetido: false
[ OK ] 5a. SW-E1 preorden: 60 35 20 45 40 80 70 75 90
[ OK ] 5b. SW-E1 inorden: 20 35 40 45 60 70 75 80 90
[ OK ] 5c. SW-E1 posorden: 20 40 45 35 75 70 90 80 60
[ OK ] 6. Insertar un código repetido: false
[ OK ] 7a. caminoBusqueda(42): [60, 35, 45, 40]
[ OK ] 7b. contarEnRango(40, 75): 5
[ OK ] 8. Insertar 42 y eliminar 20, 70 y 60: preorden 75 35 45 40 42 80 90
[ OK ] 9. Buscar, modificar y eliminar un código inexistente: null / false
[ OK ] 10. Libro con datos inválidos: IllegalArgumentException
[ OK ] 11. esABBValido() con un nieto mal ubicado: false
[ OK ] 11b. esABBValido() en un ABB construido con insertar: true
[ OK ] Extra. desdePreorden reconstruye el mismo ABB

17/17 pruebas OK
```

## Captura de ejecución: menú (opciones 1, 2, 5 y 10 con SW-E1)

```
Opción: 1
Estructura inicial cargada: 7 unidades y 13 libros.

Opción: 2
BIB Biblioteca FISEI
  SW Software
    SW-E1 Estante 1 de Software
    SW-E2 Estante 2 de Software
  TI Tecnologías de la Información
    TI-E1 Estante 1 de TI
  TEL Telecomunicaciones

Opción: 5
Altura (aristas): 2
Hojas: 4
Total de libros: 13
Código para consultar su nivel: sw
Nivel de SW: 1

Opción: 10
Estantes disponibles: SW-E1 SW-E2 TI-E1
Código del estante: SW-E1

Opción: 11
Código del estante: SW-E1
Rango: valor a: 40
Rango: valor b: 75
contarEnRango(40, 75) = 5
Código para caminoBusqueda: 42
caminoBusqueda(42) = [60, 35, 45, 40]
esABBValido() = true
```

## Reflexión personal

Con este deber aprendí a separar el modelo, la lógica y la aplicación en paquetes, y a pensar cada operación de los árboles de forma recursiva: primero el caso base y luego la llamada sobre los hijos. Lo que más me ayudó a entender el ABB fue dibujar a mano cada inserción y cada eliminación antes de ejecutar el programa, porque así pude comparar mi resultado con el del código. También entendí por qué el reto esABBValido() necesita límites mínimo y máximo y no basta con comparar cada nodo con sus hijos.

## 11. Preguntas de reflexión

### 1. ¿Por qué buscar en el ABB es O(h) y en el árbol general es O(N)? ¿Qué le pasa a h si se insertan los códigos ya ordenados?

En el ABB, cada comparación con el código del nodo actual permite descartar un subárbol completo (si el código buscado es menor se sigue por la izquierda y si es mayor, por la derecha). Así se recorre un solo camino desde la raíz hasta una hoja, es decir, como máximo h + 1 nodos, y la búsqueda cuesta O(h). En el árbol general de unidades, los hijos de un nodo no siguen ningún orden respecto al código buscado, por lo que no se puede descartar ninguna rama: en el peor caso hay que visitar las N unidades. Si los códigos se insertan ya ordenados (por ejemplo 10, 20, 30, 40, 50), cada nuevo código es mayor que todos los anteriores y siempre va a la derecha: el ABB se degenera en una lista enlazada, h pasa a ser N - 1 y la búsqueda se vuelve O(N). Con un árbol balanceado, en cambio, $h \approx \log_2 N$.

### 2. ¿Por qué, al eliminar un nodo con dos hijos, se usa el sucesor inorden y no cualquier nodo del subárbol derecho?

Porque el sucesor inorden es el menor código del subárbol derecho: es mayor que todos los códigos del subárbol izquierdo y menor o igual que los demás del derecho. Al copiar su valor en el nodo eliminado se mantiene la propiedad del ABB sin mover nada más. Si se usara otro nodo del subárbol derecho, por ejemplo el 90 al eliminar el 60 en SW-E1, los códigos 70, 75 y 80 quedarían en el subárbol derecho de un nodo con código 90 siendo menores, y el árbol dejaría de ser un ABB válido. Además, el sucesor no tiene hijo izquierdo, por lo que quitarlo de su posición original es siempre el caso fácil (hoja o un solo hijo).

### 3. ¿Cómo podría utilizarse un árbol general o un ABB en el proyecto integrador SmartETL para organizar o buscar registros?

Un árbol general serviría para organizar jerárquicamente los orígenes de datos, tal como la biblioteca organiza Áreas y Estantes: fuente → tabla o archivo → campo, de modo que se puedan recorrer y contar los elementos de cada nivel y validar qué puede colgar de qué. Un ABB serviría como índice en memoria de los registros extraídos, ordenado por una clave como el identificador o la fecha: permitiría buscar, insertar y eliminar registros sin recorrerlos todos (O(h)), obtener los registros en orden con un recorrido inorden, y consultar rangos de claves con poda, como hace contarEnRango(). Para que el índice se mantenga eficiente conviene que el árbol esté balanceado, o que los registros no lleguen ya ordenados.

## 12. Anexos

Github: https://github.com/edithtuza15-collab/Deber-Arboles-TuzaEdith

Commits: https://github.com/edithtuza15-collab/Deber-Arboles-TuzaEdith/commits/main

Pull request (desarrollo → main):
https://github.com/edithtuza15-collab/Deber-Arboles-TuzaEdith/pulls?q=is%3Apr

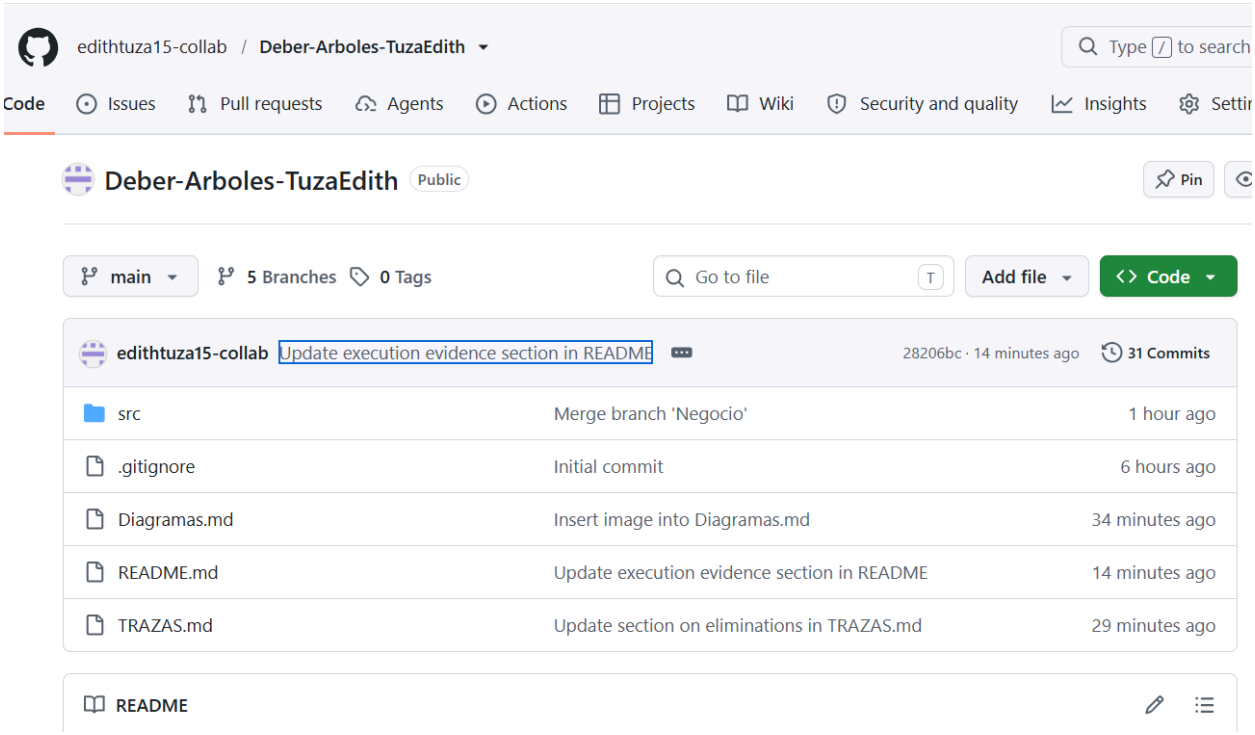
# UNIVERSIDAD TÉCNICA DE AMBATO

**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**

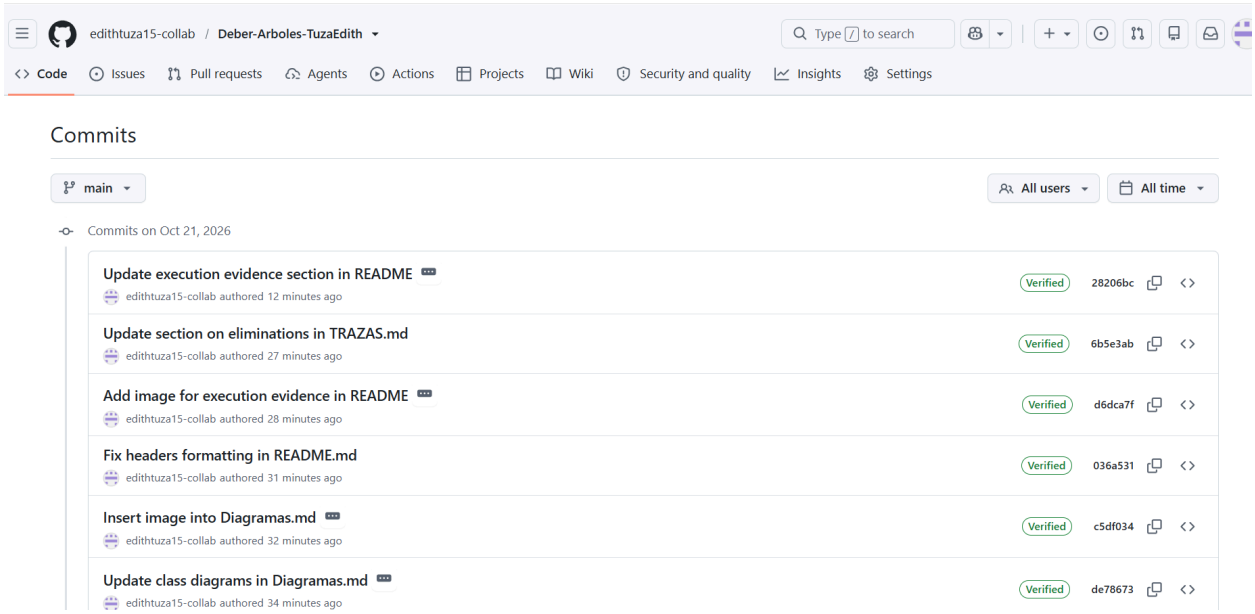
**CARRERA DE SOFTWARE**

**CICLO ACADÉMICO: AGOSTO 2026 – ENERO**

## Captura del repositorio (rama main, README.md y TRAZAS.md):




## Captura del historial de commits:



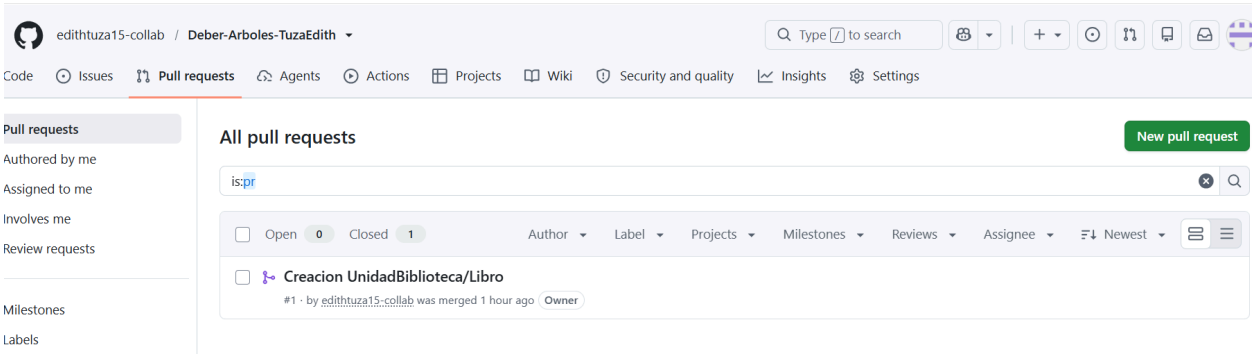

## Captura del pull request de desarrollo a main:

# UNIVERSIDAD TÉCNICA DE AMBATO

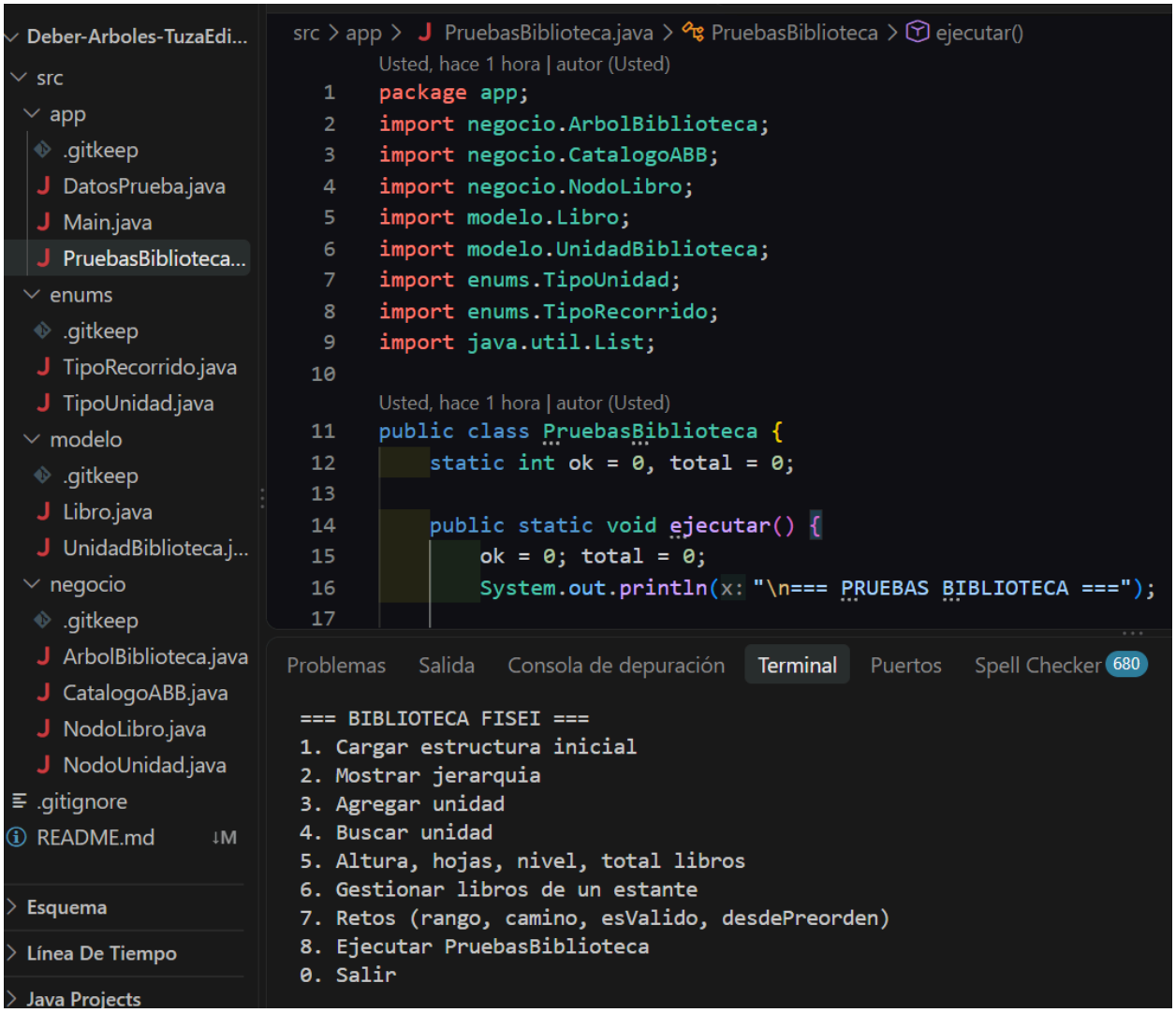
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**

**CARRERA DE SOFTWARE**

**CICLO ACADÉMICO: AGOSTO 2026 – ENERO**




**Captura de PruebasBiblioteca ejecutándose en Visual Studio Code:**



# UNIVERSIDAD TÉCNICA DE AMBATO

**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**

**CARRERA DE SOFTWARE**

**CICLO ACADÉMICO: AGOSTO 2026 – ENERO**

